# Documentation UML
# Projet VoisinPartage

10 juin 2026

## Table des matières


**1 Introduction** **2**

**2 Diagramme de cas d'utilisation** **2**
2.1 Description du diagramme . . . . . 2
2.1.1 Les acteurs identifiés . . . . . 2
2.1.2 Relations principales . . . . . 3

**3 Diagramme de classes** **4**
3.1 Description du diagramme . . . . . 4
3.1.1 Les classes principales . . . . . 4
3.1.2 Les associations . . . . . 5

**4 Diagramme de séquence : Connexion utilisateur** **6**
4.1 Description du diagramme . . . . . 6
4.1.1 Les acteurs et objets impliqués . . . . . 6
4.1.2 Scénario nominal . . . . . 6
4.1.3 Alternatives (fragment alt) . . . . . 7

**5 Diagramme de séquence : Publication d'une annonce** **8**
5.1 Description du diagramme . . . . . 8
5.1.1 Les acteurs et objets impliqués . . . . . 8
5.1.2 Scénario nominal . . . . . 8
5.1.3 Alternatives (fragment alt) . . . . . 9

**6 Conclusion** **9**

# 1 Introduction

Ce document présente les différents diagrammes UML modélisant l'application **VoisinPartage**, une plateforme de partage et d'entraide entre voisins. Les diagrammes présentés couvrent les aspects fonctionnels, structurels et dynamiques du système.

# 2 Diagramme de cas d'utilisation

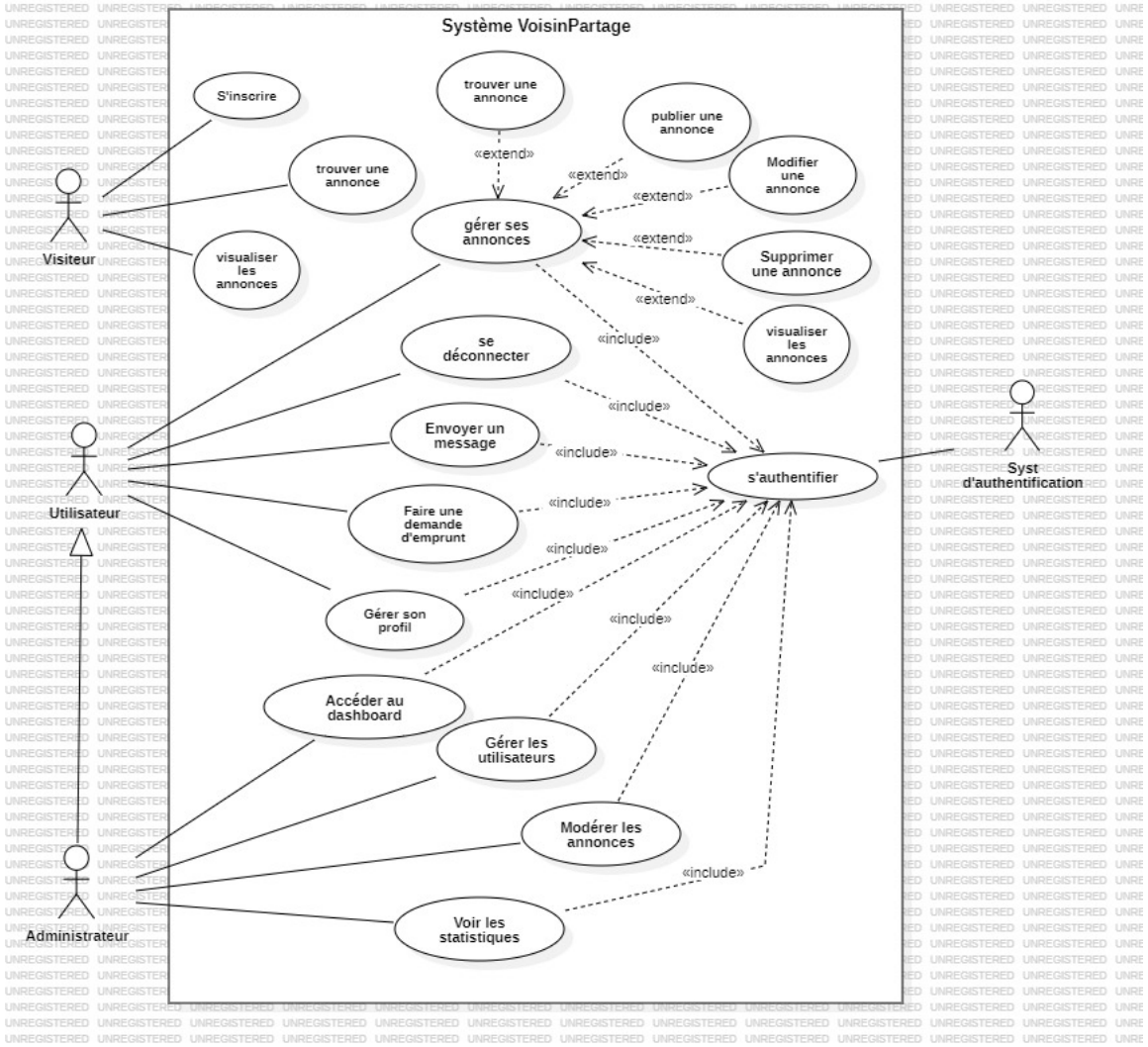



FIGURE 1 – Diagramme de cas d'utilisation du système VoisinPartage

## 2.1 Description du diagramme

Le diagramme de cas d'utilisation (figure 1) représente les interactions entre les différents acteurs et le système **VoisinPartage**.

### 2.1.1 Les acteurs identifiés

— **Visiteur** : Utilisateur non authentifié qui peut consulter les annonces, en rechercher une, s'inscrire ou visualiser la liste des annonces disponibles.

— **Utilisateur** : Hérite des fonctionnalités du visiteur et bénéficie de fonctionnalités supplémentaires telles que la gestion de ses annonces (publier, modifier, supprimer), l'envoi de messages, la demande d'emprunt, la gestion de son profil et la déconnexion.
— **Administrateur** : Hérite des droits de l'utilisateur et possède des privilèges supplémentaires comme l'accès au dashboard, la gestion des utilisateurs, la modération des annonces et la consultation des statistiques.
— **Système d'authentification** : Acteur secondaire qui prend en charge la vérification des identifiants.

### 2.1.2 Relations principales

— Les relations `«include»` indiquent que la plupart des actions sensibles (publication, envoi de message, gestion du profil, etc.) nécessitent une authentification préalable.
— Les relations `«extend»` montrent que la gestion des annonces englobe plusieurs sous-fonctionnalités optionnelles (publier, modifier, supprimer, visualiser).

# 3 Diagramme de classes

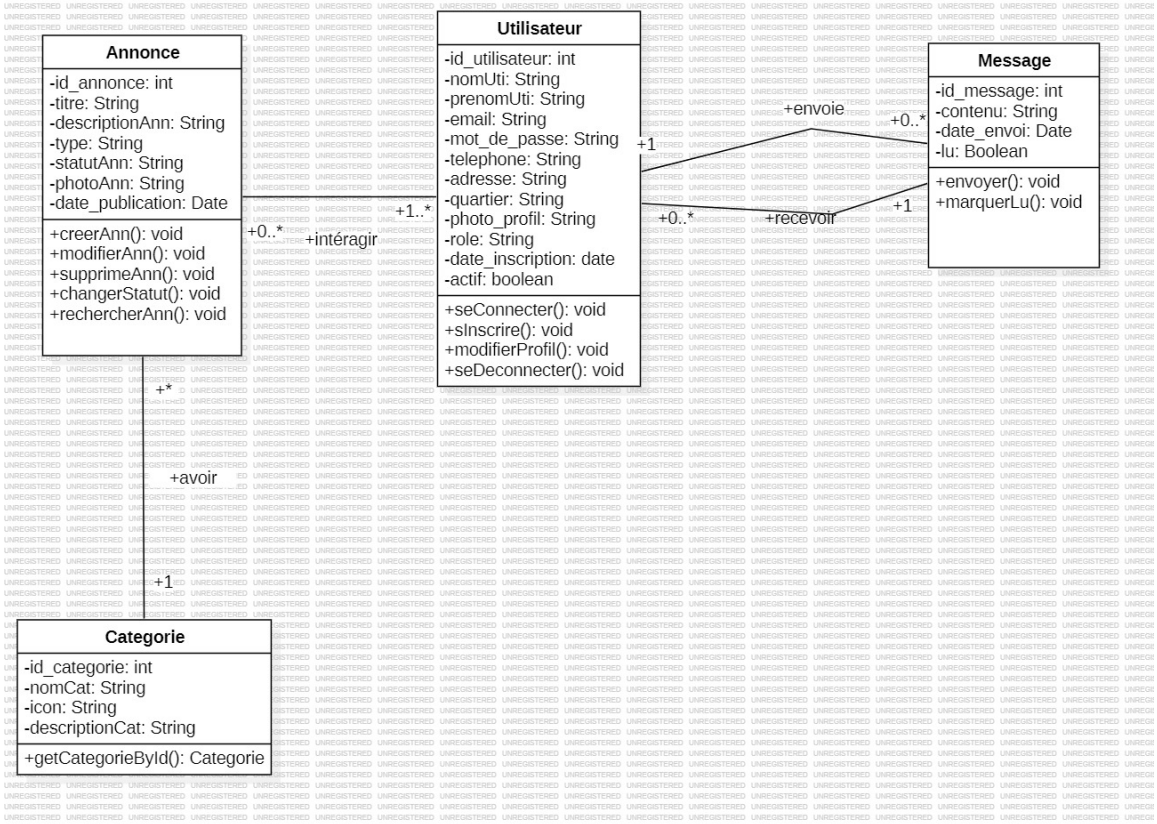



FIGURE 2 – Diagramme de classes du système VoisinPartage

## 3.1 Description du diagramme

Le diagramme de classes (figure 2) présente la structure statique du système en identifiant les classes principales, leurs attributs, leurs méthodes ainsi que les relations qui les lient.

### 3.1.1 Les classes principales

- **Utilisateur** : Représente un membre inscrit sur la plateforme. Elle contient les informations personnelles (nom, prénom, email, mot de passe, téléphone, adresse, quartier, photo de profil, rôle, date d'inscription, statut actif) et propose des méthodes telles que `seConnecter()`, `sInscrire()`, `modifierProfil()` et `seDeconnecter()`.
- **Annonce** : Représente une annonce publiée par un utilisateur. Elle comporte un identifiant, un titre, une description, un type, un statut, une photo et une date de publication. Les méthodes permettent de créer, modifier, supprimer, changer le statut ou rechercher une annonce.
- **Message** : Représente la communication entre deux utilisateurs avec un identifiant, le contenu, la date d'envoi et un état de lecture. Les méthodes `envoyer()` et `marquerLu()` gèrent le cycle de vie du message.
- **Categorie** : Permet de classer les annonces par catégorie (avec nom, icône, description) et offre la méthode `getCategorieById()`.

### 3.1.2 Les associations

— Un **Utilisateur** peut publier plusieurs **Annonces** (relation 1..*) et interagir avec 0 ou plusieurs annonces.
— Un **Utilisateur** peut envoyer et recevoir plusieurs **Messages** (relation 1..0..*).
— Chaque **Annonce** appartient à une seule **Categorie**, mais une catégorie peut regrouper plusieurs annonces.

# 4 Diagramme de séquence : Connexion utilisateur

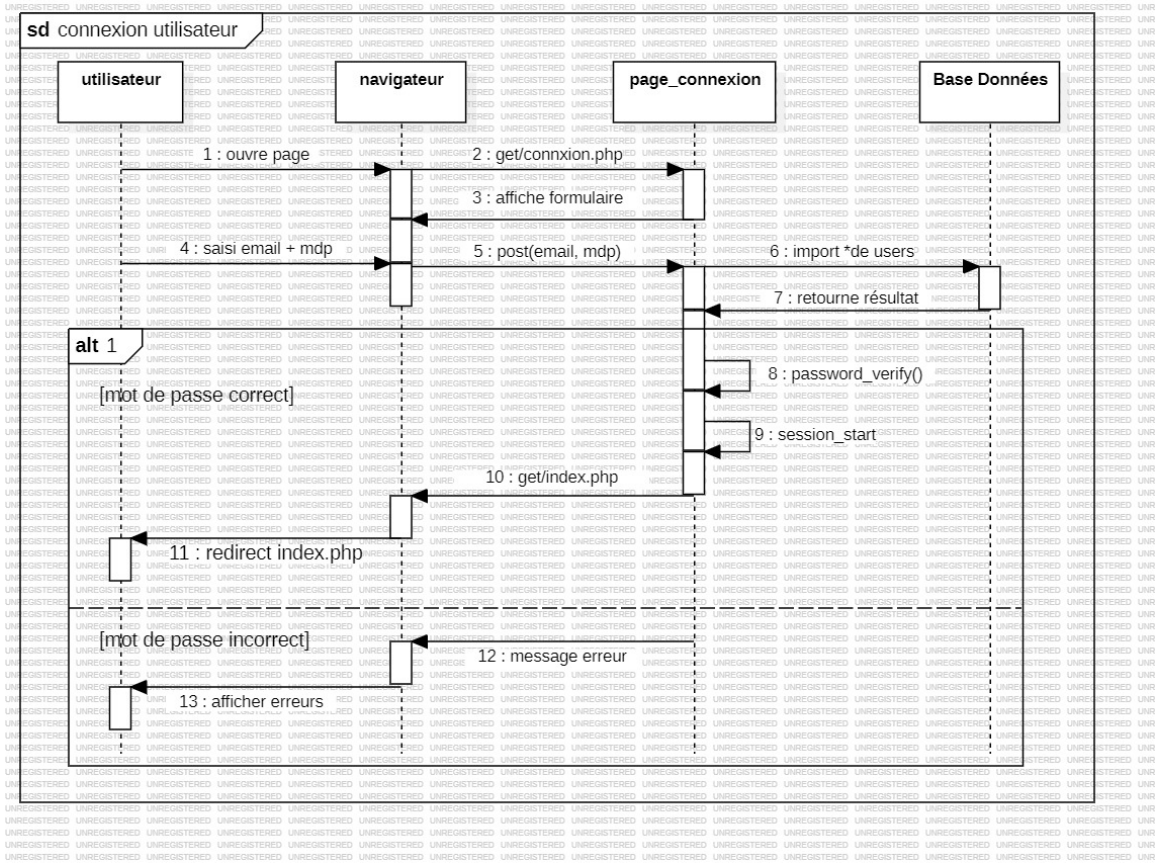



FIGURE 3 – Diagramme de séquence - Connexion d'un utilisateur

## 4.1 Description du diagramme

Le diagramme de séquence (figure 3) illustre le déroulement chronologique des échanges lors de la connexion d'un utilisateur au système.

### 4.1.1 Les acteurs et objets impliqués

- **Utilisateur** : Personne qui souhaite se connecter à la plateforme.
- **Navigateur** : Interface client utilisée pour envoyer les requêtes.
- **page_connexion** : Script PHP côté serveur qui gère la logique de connexion.
- **Base de Données** : Stocke les informations relatives aux utilisateurs.

### 4.1.2 Scénario nominal

1. L'utilisateur ouvre la page de connexion via son navigateur.
2. Le navigateur envoie une requête GET vers `connexion.php`.
3. Le formulaire de connexion s'affiche à l'utilisateur.
4. L'utilisateur saisit son email et son mot de passe.
5. Le navigateur transmet ces informations via une requête POST.

6. Le script effectue une requête vers la base de données pour vérifier l'existence de l'utilisateur.
7. La base de données retourne le résultat.

#### 4.1.3 Alternatives (fragment alt)

— **Cas 1 : Mot de passe correct**
La fonction `password_verify()` valide le mot de passe, `session_start()` initialise la session, puis l'utilisateur est redirigé vers `index.php`.
— **Cas 2 : Mot de passe incorrect**
Un message d'erreur est retourné au navigateur, qui l'affiche à l'utilisateur.

# 5 Diagramme de séquence : Publication d'une annonce

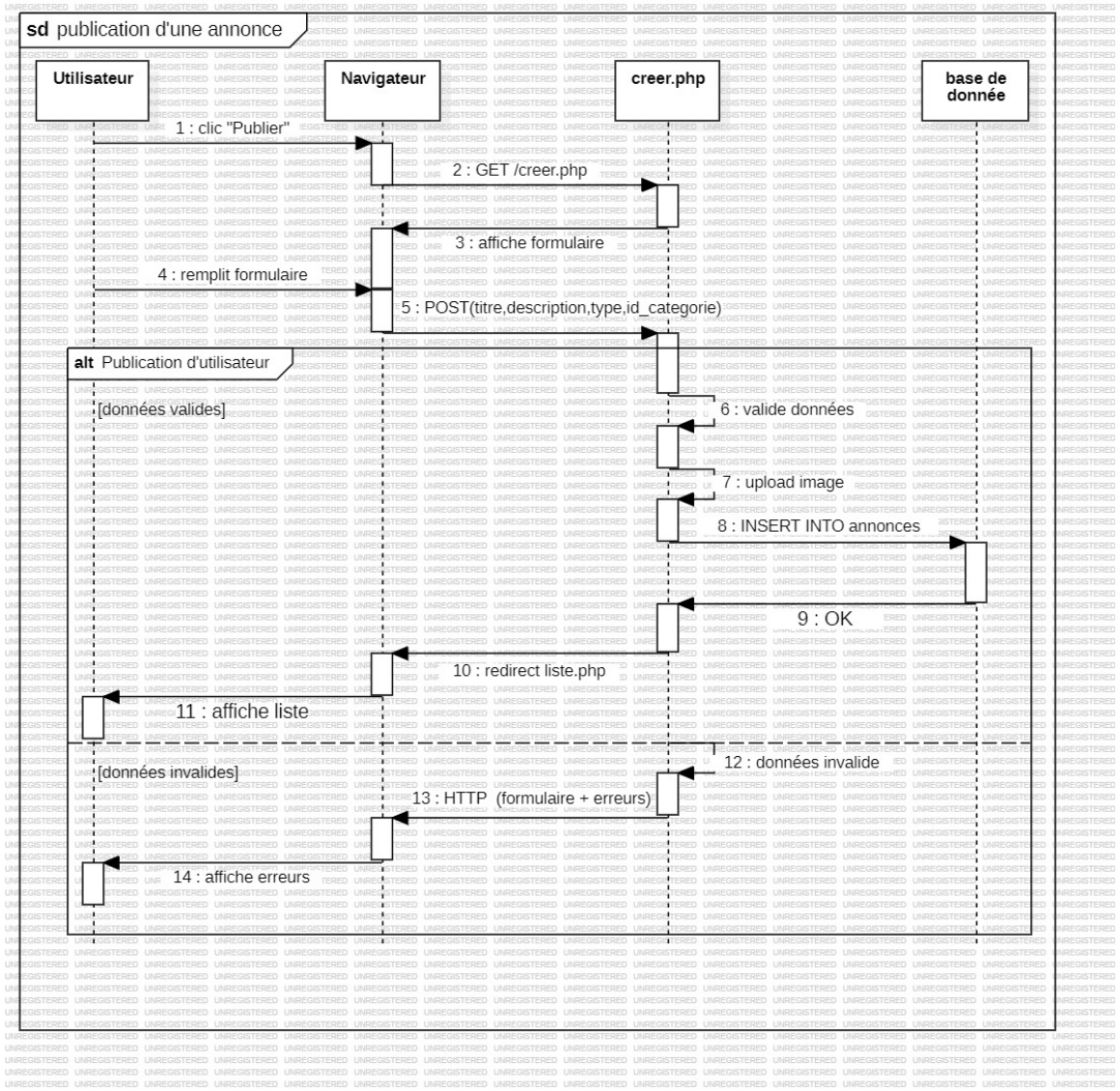



FIGURE 4 – Diagramme de séquence - Publication d'une annonce

## 5.1 Description du diagramme

Le diagramme de séquence (figure 4) décrit le processus de publication d'une nouvelle annonce par un utilisateur authentifié.

### 5.1.1 Les acteurs et objets impliqués

- **Utilisateur** : Membre authentifié qui souhaite publier une annonce.
- **Navigateur** : Interface client.
- **creer.php** : Script serveur qui gère la création des annonces.
- **Base de données** : Persiste les nouvelles annonces.

### 5.1.2 Scénario nominal

1. L'utilisateur clique sur le bouton « Publier ».

2. Le navigateur envoie une requête GET vers `creer.php`.
3. Le formulaire de création d'annonce s'affiche.
4. L'utilisateur remplit les champs (titre, description, type, catégorie, photo).
5. Le navigateur envoie les données via POST.

#### 5.1.3 Alternatives (fragment alt)

— **Cas 1 : Données valides**
Le script valide les données, télécharge l'image, puis exécute la requête `INSERT INTO annonces`. La base de données confirme l'insertion (OK), et l'utilisateur est redirigé vers la liste des annonces (`liste.php`).
— **Cas 2 : Données invalides**
Le script détecte les erreurs de saisie et renvoie une réponse HTTP contenant le formulaire et les messages d'erreur à corriger par l'utilisateur.

# 6 Conclusion

Les diagrammes présentés dans ce document offrent une vision complète du système **VoisinPartage** :

— Le **diagramme de cas d'utilisation** identifie les acteurs et leurs interactions fonctionnelles.
— Le **diagramme de classes** décrit la structure statique et les relations entre les entités du système.
— Les **diagrammes de séquence** illustrent la dynamique des interactions pour les scénarios clés (connexion et publication d'annonce).

Cette modélisation UML constitue une base solide pour le développement et la maintenance de l'application.